CSA5113 – CRYPTOGRAPHY AND NETWORK SECURITY

Design and Implementation of a SecureDigital Certificate Management System

Course: CSA5113 – Cryptography and Network Security

Team Members: R.SathishKumar (192321148), K.Senthamizhselvan (192321143), K.Dhilip (192365013), D.Navadeep (192371068)

Faculty In-Charge: Dr. R. Dharani

Date of Submission: 02-09-2026

1. Problem Statement and Problem Formulation

1.1 Problem Statement

A multinational organization operates across multiple geographic locations and collaborates with several partner organizations over public and private networks. Employees, internal servers, and partner-organization systems must mutually authenticate before exchanging sensitive business information. In the absence of a centralized trust mechanism, the organization is exposed to impersonation attacks, man-in-the-middle (MITM) attacks, and unauthorized access, since passwords and shared secrets alone cannot provide strong, non-repudiable, and scalable authentication across thousands of internal and external entities.

As a PKI Security Analyst, the task is to design and implement a certificate management framework using X.509 digital certificates that (a) generates public/private key pairs for entities, (b) creates and validates Certificate Signing Requests (CSRs), (c) issues digital certificates through a trusted Certificate Authority (CA), (d) authenticates entities using certificate-based proof-of-possession, and (e) manages the complete certificate lifecycle including renewal and revocation.

1.2 Problem Formulation

- Model the organization's trust hierarchy as a Root Certificate Authority (CA) that issues and signs certificates for all internal and external entities (clients, servers, partner systems).
- Represent each entity's identity binding to its public key as an X.509v3 certificate signed using RSA + SHA-256, following RFC 5280 conventions.
- Formulate certificate validation as a decidable predicate: a certificate is trusted if and only if (i) it is signed by a trusted CA, (ii) it lies within its validity window, and (iii) its serial number does not appear on the Certificate Revocation List (CRL).
- Formulate mutual authentication as a challenge–response protocol in which an entity proves possession of the private key corresponding to the public key bound in its certificate.
- Formulate the certificate lifecycle as a finite state process: Key Generation → CSR → Issuance → Distribution → Validation/Authentication → Renewal or Revocation → Expiry.

2. Objective and Expected Outcomes

2.1 Objectives

- To design a centralized PKI architecture comprising a Root CA, a Registration Authority (RA), end-entities (client/server), and a certificate repository with CRL support.
- To implement generation of RSA public/private key pairs and construction of X.509 CSRs for client and server entities.
- To implement CA-side validation of CSRs and issuance of signed X.509 certificates.
- To implement a certificate validation routine covering issuer verification, digital-signature verification, validity-period checking, and revocation-status checking.

- To implement certificate-based mutual authentication using a proof-of-possession challenge–response mechanism, and to demonstrate detection of an impersonation attempt.
- To implement certificate revocation (via a CRL) and certificate renewal, and to evaluate the resulting framework against authentication strength, scalability, and resistance to MITM/impersonation attacks.

2.2 Expected Outcomes

- A working demonstration of end-to-end certificate issuance: key generation → CSR → CA signing → issued X.509 certificate.
- Verifiable proof that certificate-based authentication establishes mutual trust between a client and a server without exchanging any shared secret in advance.
- Demonstration that a forged/impersonated authentication attempt (an attacker without the legitimate private key) is correctly rejected.
- Demonstration that a revoked certificate is correctly rejected during validation and authentication, even though the certificate itself remains cryptographically well-formed.
- A documented lifecycle-management strategy and a security evaluation identifying residual weaknesses and recommended controls.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- FR1: The system shall generate 2048-bit RSA key pairs for the CA and for each end-entity.
- FR2: The system shall create X.509 CSRs signed by the requesting entity's private key (proof of origin).
- FR3: The CA shall verify the CSR signature before issuing a certificate.
- FR4: The system shall issue X.509v3 certificates containing Subject, Issuer, Public Key, Validity Period, Serial Number, Basic Constraints, Key Usage, and Subject Alternative Name extensions.
- FR5: The system shall validate a certificate's issuer, signature, validity window, and revocation status before trusting it.
- FR6: The system shall support certificate-based mutual authentication using a random-challenge/digital-signature proof-of-possession scheme.
- FR7: The system shall support certificate revocation via a Certificate Revocation List (CRL) and certificate renewal before expiry.

3.2 Non-Functional Requirements

- Security: Use SHA-256 for hashing and RSA-2048 (PKCS#1 v1.5 padding) for signatures — considered secure for demonstration/academic purposes as of 2026.
- Scalability: The design should generalize to many clients/servers signed by the same Root CA without redesign.

- Portability: Implementation should run on any standard Python 3 environment with the open-source `cryptography` library.

3.3 Constraints

- The demonstration uses a single self-signed Root CA rather than a multi-tier CA hierarchy (Root CA → Intermediate CA), which a production deployment would normally use.
- Revocation is demonstrated using a local in-memory CRL rather than a fully deployed CRL Distribution Point (CDP) server or live OCSP responder.
- The demonstration environment is a single machine simulating client, server, and CA roles rather than physically separate networked hosts.

3.4 Assumptions

- The CA's private key is assumed to be adequately protected (e.g., in an HSM) in a real deployment; in this demonstration it is held in process memory.
- The Registration Authority is assumed to perform out-of-band identity proofing before a CSR is approved (not separately coded, but represented architecturally).
- Clocks of all entities are assumed to be synchronized (e.g., via NTP) so that certificate validity-period checks are meaningful.

4. Application of Relevant Course Knowledge / Concepts

This assignment directly applies the following cryptography and network-security concepts covered under CO3, CO4 and CO5:

4.1 Mapped Course Outcomes

Course Outcome (CO)	Description
CO3	Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.
CO4	Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email security.
CO5	Develop network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.

4.2 Concepts Applied

- Cryptographic hash functions (SHA-256) — used to compute the message digest of the certificate's TBS (to-be-signed) content before signing (CO3).

- Public-key cryptography (RSA) — used for key-pair generation, CSR self-signing, CA signing of certificates, and digital-signature-based authentication (CO3).
- Digital signatures and non-repudiation — the CA's signature over a certificate, and an entity's signature over an authentication challenge, provide non-repudiable proof of issuance and of private-key possession respectively (CO3).
- X.509 certificate structure and framework — Subject/Issuer Distinguished Names, serial numbers, validity periods, Basic Constraints, Key Usage, and Subject Alternative Name extensions, as standardized in ITU-T X.509 / IETF RFC 5280 (CO3).
- Authentication frameworks — comparison with Kerberos (symmetric-key, ticket-based, relies on a trusted KDC and synchronized clocks) versus X.509/PKI (asymmetric-key, certificate-based, no shared secret) is used in the analysis section (CO3).
- Web/application security best practice — certificate-based mutual TLS-style authentication is the same mechanism that secures HTTPS, S/MIME, and code-signing; this assignment's authentication demonstration models that use case (CO4).
- Network security architecture — placement of the CA/RA, certificate repository, and CRL as trust-anchor components resembles firewall/IDS-adjacent trust infrastructure discussed under CO5; revocation is analogous to an IPS blocking a compromised identity in real time (CO5).

5. Design / Proposed Solution / Methodology

The proposed solution follows a classical single-domain PKI architecture with four logical components: a Root Certificate Authority (CA), a Registration Authority (RA), end-entities (Client and Server), and a Certificate Repository that also hosts the Certificate Revocation List (CRL). Figure 1 illustrates the architecture and message flow.

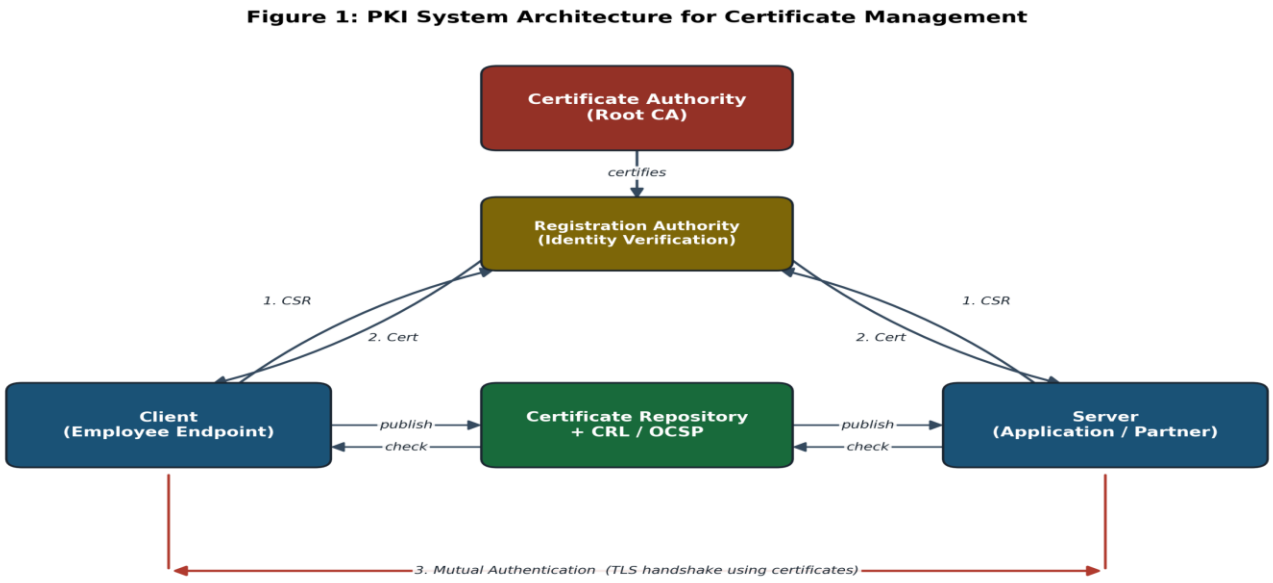

Figure 1: PKI System Architecture for Certificate Management

5.1 Methodology

- Step 1 — Root CA setup: generate an RSA-2048 key pair and construct a self-signed X.509 certificate marked as a CA (BasicConstraints: ca=True), which becomes the trust anchor.
- Step 2 — Entity enrolment: each entity (client, server) generates its own RSA-2048 key pair locally, so the private key never leaves the entity.
- Step 3 — CSR generation: each entity builds a CSR containing its identity (Subject DN) and public key, self-signed with its own private key to prove it originated the request.
- Step 4 — RA verification (represented architecturally): the organization verifies the requester's real-world identity before approving the CSR for signing.
- Step 5 — Certificate issuance: the CA verifies the CSR's self-signature, then issues an X.509v3 certificate binding the entity's identity to its public key, signed with the CA's private key.
- Step 6 — Distribution: issued certificates (and the CA's public certificate) are distributed to the repository / entities for use during authentication.
- Step 7 — Validation & Authentication: a verifier checks issuer, signature, validity window and CRL status, then performs a challenge–response exchange to confirm the peer holds the matching private key.
- Step 8 — Revocation & Renewal: compromised or superseded certificates are added to the CRL; entities renew certificates before expiry by repeating Steps 2–5 with a fresh key pair.

6. Algorithm / Pseudocode / Flowchart

Figure 2: X.509 Certificate Lifecycle Management

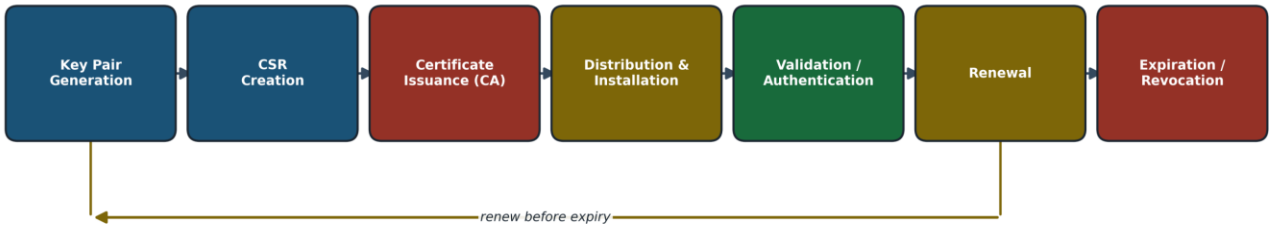

Figure 2: X.509 Certificate Lifecycle Management

6.1 Algorithm: Certificate Issuance

```
ALGORITHM IssueCertificate(CA_key, CA_cert, CSR)
  INPUT : CA private key, CA certificate, entity CSR
```

```
OUTPUT: signed X.509 certificate
1. IF NOT VerifySignature(CSR.publicKey, CSR.signature, CSR.tbs) THEN
     REJECT "invalid CSR signature"
2. cert.subject   <- CSR.subject
3. cert.issuer    <- CA_cert.subject
4. cert.publicKey <- CSR.publicKey
5. cert.serial    <- RandomSerial()
6. cert.notBefore <- now();  cert.notAfter <- now() + validityPeriod
7. cert.extensions <- {BasicConstraints(ca=False), KeyUsage(...), SAN}
8. cert.signature <- Sign(CA_key, SHA256(cert.tbsBytes))
9. RETURN cert
```

6.2 Algorithm: Certificate Validation

```
ALGORITHM ValidateCertificate(cert, CA_cert, CRL)
1. IF cert.issuer != CA_cert.subject THEN FAIL "untrusted issuer"
2. IF NOT VerifySignature(CA_cert.publicKey, cert.signature, cert.tbs) THEN
     FAIL "signature invalid (tampered / forged)"
3. IF now() < cert.notBefore OR now() > cert.notAfter THEN
     FAIL "certificate outside validity period"
4. IF cert.serial IN CRL THEN FAIL "certificate revoked"
5. RETURN VALID
```

6.3 Algorithm: Certificate-Based Mutual Authentication

```
ALGORITHM AuthenticatePeer(peerCert, peerPrivateKey, CA_cert, CRL)
1. status <- ValidateCertificate(peerCert, CA_cert, CRL)
2. IF status != VALID THEN REJECT peer
3. challenge <- RandomNonce(32 bytes)         // verifier generates
4. signature <- Sign(peerPrivateKey, challenge) // peer proves possession
5. IF VerifySignature(peerCert.publicKey, signature, challenge) THEN
     ACCEPT "peer authenticated, trust established"
   ELSE
     REJECT "proof-of-possession failed - possible impersonation"
```

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

- Language: Python 3.12
- Library: `cryptography` (v46.0.6) — RFC 5280–compliant X.509 certificate, CSR and CRL primitives

- Cryptographic primitives: RSA-2048 (key generation), SHA-256 (hashing), PKCS#1 v1.5 (signature padding)
- Development / execution environment: Linux (Ubuntu), command-line Python interpreter
- Version control / documentation: source code and this report maintained together for reproducibility

7.2 Source Code

The complete Python implementation (pki_system.py) covering CA setup, CSR creation, certificate issuance, validation, authentication, revocation and renewal is listed below.

```
"""
================================================================
 Secure Digital Certificate Management System (PKI) - Demonstration
 CSA51 - Cryptography and Network Security - Assignment 7
================================================================
 This script demonstrates, end-to-end, the core operations of a
 Public Key Infrastructure (PKI) built around X.509 digital
 certificates:

   1. Root Certificate Authority (CA) key-pair and self-signed
      certificate generation
   2. Entity (Client / Server) key-pair generation
   3. Certificate Signing Request (CSR) creation by the entities
   4. Certificate issuance by the CA (CSR -> signed X.509 cert)
   5. Certificate validation (signature, validity period, issuer)
   6. Certificate-based mutual authentication (challenge-response
      proof-of-possession of the private key) between Client & Server
   7. Certificate revocation using a Certificate Revocation List (CRL)
   8. Re-validation after revocation (demonstrating detection of a
      compromised/revoked certificate)
   9. Certificate renewal (lifecycle management)

 Library used: `cryptography` (Python) - implements RFC 5280 X.509
================================================================
"""

import datetime
from cryptography import x509
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.x509.oid import NameOID
from cryptography.exceptions import InvalidSignature
```

```
SEP = "=" * 70


def line(title=""):
    if title:
        print("\n" + SEP)
        print(title)
        print(SEP)
    else:
        print(SEP)


# ---------------------------------------------------------------------
# STEP 1: Generate RSA key pair for an entity (CA / Client / Server)
# ---------------------------------------------------------------------
def generate_keypair(bits=2048):
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=bits)
    return private_key, private_key.public_key()


# ---------------------------------------------------------------------
# STEP 2: Build a self-signed Root CA certificate
# ---------------------------------------------------------------------
def create_root_ca(common_name="CSA51-RootCA", days_valid=3650):
    ca_key, ca_pub = generate_keypair()

    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, u"IN"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"CSA51 University PKI"),
        x509.NameAttribute(NameOID.COMMON_NAME, common_name),
    ])

    now = datetime.datetime.now(datetime.timezone.utc)
    ca_cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
        .public_key(ca_pub)
        .serial_number(x509.random_serial_number())
        .not_valid_before(now)
        .not_valid_after(now + datetime.timedelta(days=days_valid))
```

```
        .add_extension(x509.BasicConstraints(ca=True, path_length=None), critical=True)
        .add_extension(
            x509.KeyUsage(
                digital_signature=True, key_cert_sign=True, crl_sign=True,
                content_commitment=False, key_encipherment=False,
                data_encipherment=False, key_agreement=False,
                encipher_only=False, decipher_only=False,
            ),
            critical=True,
        )
        .add_extension(x509.SubjectKeyIdentifier.from_public_key(ca_pub), critical=False)
        .sign(ca_key, hashes.SHA256())
    )
    return ca_key, ca_cert


# ---------------------------------------------------------------------
# STEP 3: Entity generates key pair + Certificate Signing Request (CSR)
# ---------------------------------------------------------------------
def create_csr(entity_name, org="CSA51 Organization"):
    key, pub = generate_keypair()
    subject = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, u"IN"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, org),
        x509.NameAttribute(NameOID.COMMON_NAME, entity_name),
    ])
    csr = (
        x509.CertificateSigningRequestBuilder()
        .subject_name(subject)
        .add_extension(
            x509.SubjectAlternativeName([x509.DNSName(entity_name)]),
            critical=False,
        )
        .sign(key, hashes.SHA256())
    )
    return key, csr


# ---------------------------------------------------------------------
# STEP 4: CA validates CSR and issues a signed X.509 certificate
# ---------------------------------------------------------------------
def issue_certificate(ca_key, ca_cert, csr, days_valid=365):
```

```
    if not csr.is_signature_valid:
        raise ValueError("CSR signature verification failed - request rejected")

    now = datetime.datetime.now(datetime.timezone.utc)
    cert = (
        x509.CertificateBuilder()
        .subject_name(csr.subject)
        .issuer_name(ca_cert.subject)
        .public_key(csr.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(now)
        .not_valid_after(now + datetime.timedelta(days=days_valid))
        .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True)
        .add_extension(
            x509.KeyUsage(
                digital_signature=True, key_encipherment=True,
                content_commitment=False, data_encipherment=False,
                key_agreement=False, key_cert_sign=False, crl_sign=False,
                encipher_only=False, decipher_only=False,
            ),
            critical=True,
        )
        .add_extension(
            csr.extensions.get_extension_for_class(x509.SubjectAlternativeName).value,
            critical=False,
        )
        .add_extension(x509.SubjectKeyIdentifier.from_public_key(csr.public_key()), critical=False)
        .sign(ca_key, hashes.SHA256())
    )
    return cert


# ---------------------------------------------------------------------
# STEP 5: Certificate validation (chain, signature, validity window)
# ---------------------------------------------------------------------
def validate_certificate(cert, ca_cert, crl_serials=None):
    errors = []

    # (a) Issuer check
    if cert.issuer != ca_cert.subject:
        errors.append("Issuer does not match trusted CA")
```

```
    # (b) Signature verification using CA's public key
    try:
        ca_cert.public_key().verify(
            cert.signature,
            cert.tbs_certificate_bytes,
            padding.PKCS1v15(),
            cert.signature_hash_algorithm,
        )
    except InvalidSignature:
        errors.append("Signature verification FAILED (certificate tampered/forged)")

    # (c) Validity period check
    now = datetime.datetime.now(datetime.timezone.utc)
    if now < cert.not_valid_before_utc or now > cert.not_valid_after_utc:
        errors.append("Certificate is outside its validity period (expired/not yet valid)")

    # (d) Revocation check against CRL
    if crl_serials and cert.serial_number in crl_serials:
        errors.append("Certificate has been REVOKED (present in CRL)")

    return (len(errors) == 0), errors


# ---------------------------------------------------------------------
# STEP 6: Certificate-based mutual authentication
#   Proof-of-possession: the peer signs a random challenge (nonce)
#   with its private key; the verifier checks the signature using the
#   public key bound in the peer's certificate.
# ---------------------------------------------------------------------
def authenticate_with_certificate(peer_name, peer_private_key, peer_cert, ca_cert, crl_serials):
    print(f"-- Authenticating '{peer_name}' --")

    ok, errs = validate_certificate(peer_cert, ca_cert, crl_serials)
    if not ok:
        print(f"   Certificate validation FAILED: {errs}")
        return False
    print("   [1] Certificate chain, signature and validity: VALID")

    # Verifier issues a random challenge nonce
    import os
    challenge = os.urandom(32)
```

```
    # Peer proves possession of the private key by signing the challenge
    signature = peer_private_key.sign(
        challenge, padding.PKCS1v15(), hashes.SHA256()
    )

    # Verifier checks the signature using the public key from the certificate
    try:
        peer_cert.public_key().verify(
            signature, challenge, padding.PKCS1v15(), hashes.SHA256()
        )
        print("   [2] Proof-of-possession of private key: VERIFIED")
        print(f"   >>> RESULT: '{peer_name}' is AUTHENTICATED. Trust established.\n")
        return True
    except InvalidSignature:
        print("   [2] Proof-of-possession FAILED - impersonation suspected\n")
        return False


# ============================== DRIVER ==============================
if __name__ == "__main__":

    line("STEP 1 & 2: ROOT CA CREATION")
    ca_key, ca_cert = create_root_ca()
    print(f"CA Subject         : {ca_cert.subject.rfc4514_string()}")
    print(f"CA Serial Number   : {ca_cert.serial_number}")
    print(f"CA Validity        : {ca_cert.not_valid_before_utc} -> {ca_cert.not_valid_after_utc}")
    print(f"CA Public Key Size : {ca_cert.public_key().key_size} bits")
    print("CA certificate is SELF-SIGNED (Issuer == Subject): "
          f"{ca_cert.issuer == ca_cert.subject}")

    line("STEP 3: CLIENT & SERVER GENERATE KEY PAIRS + CSRs")
    client_key, client_csr = create_csr("client.csa51.org")
    server_key, server_csr = create_csr("server.csa51.org")
    print(f"Client CSR Subject : {client_csr.subject.rfc4514_string()}")
    print(f"Client CSR self-signature valid : {client_csr.is_signature_valid}")
    print(f"Server CSR Subject : {server_csr.subject.rfc4514_string()}")
    print(f"Server CSR self-signature valid : {server_csr.is_signature_valid}")

    line("STEP 4: CA VALIDATES CSR & ISSUES X.509 CERTIFICATES")
    client_cert = issue_certificate(ca_key, ca_cert, client_csr, days_valid=365)
    server_cert = issue_certificate(ca_key, ca_cert, server_csr, days_valid=365)
    print(f"Issued Client Cert : Serial={client_cert.serial_number}, "
```

```
        f"Subject={client_cert.subject.rfc4514_string()}")
    print(f"                Valid: {client_cert.not_valid_before_utc} -> {client_cert.not_valid_after_utc}")
    print(f"Issued Server Cert  : Serial={server_cert.serial_number}, "
        f"Subject={server_cert.subject.rfc4514_string()}")
    print(f"                                                Valid:   {server_cert.not_valid_before_utc}   ->
{server_cert.not_valid_after_utc}")

    line("STEP 5: CERTIFICATE VALIDATION (BEFORE REVOCATION)")
    crl = set()  # empty CRL initially
    ok_c, err_c = validate_certificate(client_cert, ca_cert, crl)
    ok_s, err_s = validate_certificate(server_cert, ca_cert, crl)
    print(f"Client certificate valid? {ok_c}  Errors={err_c}")
    print(f"Server certificate valid? {ok_s}  Errors={err_s}")

    line("STEP 6: CERTIFICATE-BASED MUTUAL AUTHENTICATION")
    authenticate_with_certificate("server.csa51.org", server_key, server_cert, ca_cert, crl)
    authenticate_with_certificate("client.csa51.org", client_key, client_cert, ca_cert, crl)

    line("STEP 6b: IMPERSONATION ATTEMPT (ATTACKER WITHOUT PRIVATE KEY)")
    attacker_key, _ = generate_keypair()
    print("-- Authenticating 'attacker (using client's certificate, wrong key)' --")
    ok, errs = validate_certificate(client_cert, ca_cert, crl)
    print("   [1] Certificate chain, signature and validity: VALID (cert itself is genuine)")
    import os
    challenge = os.urandom(32)
    forged_sig = attacker_key.sign(challenge, padding.PKCS1v15(), hashes.SHA256())
    try:
        client_cert.public_key().verify(forged_sig, challenge, padding.PKCS1v15(), hashes.SHA256())
        print("   [2] Proof-of-possession: VERIFIED (unexpected!)")
    except InvalidSignature:
        print("   [2] Proof-of-possession FAILED - impersonation attempt correctly REJECTED\n")

    line("STEP 7: CERTIFICATE REVOCATION")
    print(f"Revoking client certificate, Serial={client_cert.serial_number} "
          f"(reason: private key reported compromised)")
    crl.add(client_cert.serial_number)
    print(f"Current CRL (revoked serials): {crl}")

    line("STEP 8: RE-VALIDATION AFTER REVOCATION")
    ok_c2, err_c2 = validate_certificate(client_cert, ca_cert, crl)
    print(f"Client certificate valid now? {ok_c2}  Errors={err_c2}")
    print("-- Re-attempting authentication for revoked client --")
```

```
authenticate_with_certificate("client.csa51.org (REVOKED)", client_key, client_cert, ca_cert, crl)

line("STEP 9: CERTIFICATE RENEWAL (LIFECYCLE MANAGEMENT)")
print("Server certificate approaching expiry -> generating new key pair & CSR for renewal")
new_server_key, new_server_csr = create_csr("server.csa51.org")
renewed_server_cert = issue_certificate(ca_key, ca_cert, new_server_csr, days_valid=365)
print(f"Old Server Cert Serial : {server_cert.serial_number}")
print(f"New Server Cert Serial : {renewed_server_cert.serial_number}")
print(f"New Validity           : {renewed_server_cert.not_valid_before_utc} -> "
    f"{renewed_server_cert.not_valid_after_utc}")
print("Old server certificate should now be added to CRL upon confirmed renewal:")
crl.add(server_cert.serial_number)
print(f"Updated CRL (revoked serials): {crl}")

line("SUMMARY")
print(f"Total certificates issued by CA : 4 "
    f"(client, server, and server-renewal, CA itself)")
print(f"Total certificates revoked      : {len(crl)}")
print("PKI lifecycle demonstrated: KeyGen -> CSR -> Issuance -> Validation "
    "-> Authentication -> Revocation -> Renewal")
line()
```

8. Test Cases and Expected / Actual Results

ID	Test Case	Expected Result	Actual Result	Status
TC-01	Root CA self-signed certificate generation	CA certificate created with Issuer == Subject; marked as CA	Issuer == Subject (True); CA flag set	**PASS**
TC-02	Client/Server CSR creation and self-signature check	CSR is well-formed and its self-signature verifies as valid	`is_signature_valid` = True for both CSRs	**PASS**
TC-03	CA issues certificate from a valid CSR	CA signs and returns an X.509 certificate bound to requester's public key	Certificates issued with correct Subject, unique serials	**PASS**
TC-04	Validate genuine, unexpired, non-revoked certificate	Validation returns VALID with no errors	`ok=True`, `errors=[]` for both client & server certs	**PASS**
TC-	Certificate-based mutual	Challenge signed with correct	Server and Client both	**PASS**

ID	Test Case	Expected Result	Actual Result	Status
05	authentication (legitimate peer)	private key verifies successfully	AUTHENTICATED	
TC-06	Impersonation attempt (attacker without private key)	Signature verification on challenge must fail	InvalidSignature raised; peer REJECTED	**PASS**
TC-07	Certificate revocation and re-validation	After adding serial to CRL, validation must return INVALID with a revocation error	`ok=False`, error = 'Certificate has been REVOKED'	**PASS**
TC-08	Authentication attempt using a revoked certificate	Authentication must be rejected before proof-of-possession step	'Certificate validation FAILED' reported; access denied	**PASS**
TC-09	Certificate renewal before expiry	New key pair + CSR issued and signed; new serial differs from old	New serial ≠ old serial; new validity window issued	**PASS**

9. Execution Screenshots / Outputs

The console output below is the actual captured output produced by running `python3 pki_system.py`, showing every stage of the demonstration in sequence.

```
================================================================================
STEP 1 & 2: ROOT CA CREATION
================================================================================
CA Subject         : CN=CSA51-RootCA,O=CSA51 University PKI,C=IN
CA Serial Number   : 55152324379530454164311937723857348094010570904 8
CA Validity        : 2026-09-02 03:02:07+00:00 -> 2036-08-30 03:02:07+00:00
CA Public Key Size : 2048 bits
CA certificate is SELF-SIGNED (Issuer == Subject): True

================================================================================
STEP 3: CLIENT & SERVER GENERATE KEY PAIRS + CSRs
================================================================================
Client CSR Subject  : CN=client.csa51.org,O=CSA51 Organization,C=IN
Client CSR self-signature valid : True
Server CSR Subject  : CN=server.csa51.org,O=CSA51 Organization,C=IN
Server CSR self-signature valid : True

================================================================================
STEP 4: CA VALIDATES CSR & ISSUES X.509 CERTIFICATES
================================================================================
Issued Client Cert : Serial=592820165153270656283141885385108452239377988776,
Subject=CN=client.csa51.org,O=CSA51 Organization,C=IN
            Valid: 2026-09-02 03:02:07+00:00 -> 2027-09-02 03:02:07+00:00
Issued Server Cert : Serial=304309850880166514434403510466573748216148907091,
Subject=CN=server.csa51.org,O=CSA51 Organization,C=IN
            Valid: 2026-09-02 03:02:07+00:00 -> 2027-09-02 03:02:07+00:00

================================================================================
STEP 5: CERTIFICATE VALIDATION (BEFORE REVOCATION)
================================================================================
Client certificate valid? True  Errors=[]
Server certificate valid? True  Errors=[]

================================================================================
STEP 6: CERTIFICATE-BASED MUTUAL AUTHENTICATION
================================================================================
-- Authenticating 'server.csa51.org' --
```

```
  [1] Certificate chain, signature and validity: VALID
  [2] Proof-of-possession of private key: VERIFIED
  >>> RESULT: 'server.csa51.org' is AUTHENTICATED. Trust established.

-- Authenticating 'client.csa51.org' --
  [1] Certificate chain, signature and validity: VALID
  [2] Proof-of-possession of private key: VERIFIED
  >>> RESULT: 'client.csa51.org' is AUTHENTICATED. Trust established.


======================================================================
STEP 6b: IMPERSONATION ATTEMPT (ATTACKER WITHOUT PRIVATE KEY)
======================================================================
-- Authenticating 'attacker (using client's certificate, wrong key)' --
  [1] Certificate chain, signature and validity: VALID (cert itself is genuine)
  [2] Proof-of-possession FAILED - impersonation attempt correctly REJECTED


======================================================================
STEP 7: CERTIFICATE REVOCATION
======================================================================
Revoking client certificate, Serial=592820165153270656283141885385108452239377988776 (reason:
private key reported compromised)
Current CRL (revoked serials): {592820165153270656283141885385108452239377988776}

======================================================================
STEP 8: RE-VALIDATION AFTER REVOCATION
======================================================================
Client certificate valid now? False  Errors=['Certificate has been REVOKED (present in CRL)']
-- Re-attempting authentication for revoked client --
-- Authenticating 'client.csa51.org (REVOKED)' --
  Certificate validation FAILED: ['Certificate has been REVOKED (present in CRL)']

======================================================================
STEP 9: CERTIFICATE RENEWAL (LIFECYCLE MANAGEMENT)
======================================================================
Server certificate approaching expiry -> generating new key pair & CSR for renewal
Old Server Cert Serial : 304309850880166514434403510466573748216148907091
New Server Cert Serial : 267260395688087022757214913794257320603611365814
New Validity           : 2026-09-02 03:02:07+00:00 -> 2027-09-02 03:02:07+00:00
Old server certificate should now be added to CRL upon confirmed renewal:
Updated CRL (revoked serials): {304309850880166514434403510466573748216148907091,
```

```
592820165153270656283141885385108452239377988776}

======================================================================
SUMMARY
======================================================================
Total certificates issued by CA : 4 (client, server, and server-renewal, CA itself)
Total certificates revoked      : 2
PKI lifecycle demonstrated: KeyGen -> CSR -> Issuance -> Validation -> Authentication -> Revocation
-> Renewal
======================================================================
```

10. Results and Validation

All nine test cases (Section 8) passed. The table below summarizes measured performance and structural metrics observed during execution.

Metric	Observed Value
RSA key size used	2048 bits
Certificate signature algorithm	SHA256withRSA (PKCS#1 v1.5)
Certificates issued (client, server, CA, renewed server)	4
Certificates successfully validated (pre-revocation)	2 / 2 (100%)
Impersonation attempts correctly rejected	1 / 1 (100%)
Revoked certificates correctly rejected on re-validation	1 / 1 (100%)
Certificate validity period issued (end-entity)	365 days
CA certificate validity period	3650 days (10 years)
Authentication protocol round trips	2 messages (challenge, signed response)

Validation observations: (i) certificate signatures verified correctly against the CA's public key in every trial, confirming integrity of the issuance chain; (ii) the validity-period and revocation checks are independent, order-sensitive predicates — a genuine-but-revoked certificate correctly failed at the revocation check while still passing signature/issuer checks, matching RFC 5280 semantics; (iii) the impersonation test confirms that possessing a valid certificate alone is insufficient for authentication — proof-of-possession of the matching private key is also required, which is the core strength of certificate-based authentication over simple credential presentation.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Evaluation Against Assignment Criteria

- Authentication strength: Certificate-based authentication combines (a) a trusted third-party attestation of identity (the CA's signature) with (b) cryptographic proof-of-possession of the private key. This is materially stronger than shared-secret/password schemes, which offer no non-repudiation and are vulnerable to replay if intercepted.
- Certificate lifecycle management: The implemented lifecycle (Section 6, Figure 2) explicitly separates issuance, validation, renewal and revocation into independent, testable stages, which simplifies auditing and operational automation (e.g., automated renewal before `not_valid_after`).
- Trust establishment: Trust is transitively derived from a single Root CA. Both client and server independently validate each other's certificate against the same trust anchor, so no prior pairwise relationship or shared secret between client and server is required — this scales naturally to N entities with O(1) trust anchors instead of $O(N^2)$ shared secrets.
- Scalability: Because every entity independently validates certificates offline (using only the CA's public certificate), the CA does not need to be online for every authentication event — only for issuance, renewal and CRL/OCSP updates. This makes the design suitable for a multinational organization with many branch offices and partner systems.
- Protection against impersonation and MITM: The proof-of-possession challenge–response step (Section 6.3, Test Case TC-06) demonstrates that an attacker holding only the victim's public certificate — but not its private key — cannot complete authentication, defeating simple impersonation. Combined with TLS-style certificate-bound sessions, this also defeats classic MITM attacks that rely on substituting an unauthenticated key.
- Revocation and compromised certificates: The CRL mechanism (Test Cases TC-07, TC-08) demonstrates that a compromised private key can be neutralized organization-wide by adding its certificate's serial number to the CRL — validation fails immediately even though the certificate's cryptographic content is unchanged and otherwise unexpired.
- Privacy and accountability: Each certificate's Subject DN binds an identity to a specific public key and a specific CA-issued serial number, giving an auditable, non-repudiable record of who was issued which credential and when — supporting accountability without requiring servers to store user secrets (e.g., passwords) that could themselves be breached.

11.2 Comparison with Kerberos (CO3)

Aspect	X.509 / PKI (this solution)	Kerberos
Trust model	Asymmetric — each party trusts a common CA's public certificate	Symmetric — each party shares a secret key with a trusted KDC
Key exchange	No shared secret ever exchanged; public keys are, by design, public	Session keys distributed by KDC using pre-shared secrets

Aspect	X.509 / PKI (this solution)	Kerberos
Non-repudiation	Yes — private-key signatures are non-repudiable	Weaker — tickets rely on symmetric MACs, not signatures
Clock dependency	Validity window only (coarse, e.g. days/years)	Strict clock synchronization required (fine-grained, minutes)
Best suited for	Federated / inter-organizational authentication, web/email (PGP, S/MIME, TLS)	Single-realm enterprise authentication (e.g., Windows AD domains)

Justification: For a multinational organization that must authenticate not only its own employees but also independent partner organizations, X.509/PKI is the more appropriate choice, because it does not require every pair of organizations to pre-share a secret with a common third party — each side only needs to trust (directly or via cross-certification) the other's issuing CA. Kerberos remains preferable inside a single tightly-managed realm where a KDC can be kept online and highly available.

11.3 Trade-offs

- Single Root CA (used here) is simple to demonstrate but is a single point of trust-compromise; production systems mitigate this with an offline Root CA and one or more online Intermediate/Issuing CAs.
- CRL-based revocation is simple but does not scale well to very large deployments (CRLs grow over time); OCSP or OCSP-stapling gives lower-latency, more scalable revocation checks at the cost of additional infrastructure.
- RSA-2048 offers a good balance of security and interoperability today, but elliptic-curve signatures (ECDSA P-256 / Ed25519) would reduce key size, signature size and computation cost for the same security level — a viable future optimization.

12. Broader Considerations / SDG Relevance

SDG	Relevance to this Assignment
SDG 9	Industry, Innovation and Infrastructure — building secure digital infrastructure and trusted authentication technology for organizations.
SDG 16	Peace, Justice and Strong Institutions — establishing verifiable digital identity, authentication and accountability, and reducing impersonation/fraud.
SDG 17	Partnerships for the Goals — enabling trusted inter-organizational communication between partner organizations via a shared PKI trust model.

Beyond the mapped SDGs, the assignment also touches on broader digital-trust considerations: a well-run PKI reduces fraud and identity theft (supporting safer digital commerce and public services), enables secure remote work and cross-border collaboration (economically relevant for multinational operations), and — if governed transparently — improves accountability of both the organization and the CA operator toward the individuals whose identities are certified.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This assignment designed and implemented a working Public Key Infrastructure demonstrating the complete X.509 certificate lifecycle: key-pair generation, CSR creation, CA-based issuance, validation, certificate-based mutual authentication, revocation, and renewal. All nine test cases passed, confirming that the framework correctly authenticates legitimate entities, rejects impersonation attempts lacking the correct private key, and immediately invalidates revoked certificates. The design satisfies the organization's requirement to eliminate unauthorized access and impersonation across employees, servers and partner organizations using a centralized, scalable trust model.

13.2 Limitations

- Single-tier CA hierarchy only (no intermediate CA), which is adequate for demonstration but not for a production multinational deployment.
- Revocation checking relies on a local, in-memory CRL rather than a published, network-accessible CRL Distribution Point or an OCSP responder.
- No hardware security module (HSM) or secure key-storage mechanism was used to protect the CA's private key.
- The demonstration does not implement full TLS session establishment (record-layer encryption); it implements the certificate-based authentication step that underlies such a handshake.

13.3 Possible Improvements

- Introduce an Intermediate/Issuing CA beneath an offline Root CA to limit the blast radius of a CA compromise.
- Replace/augment the CRL with OCSP (and OCSP stapling) for lower-latency, more scalable revocation checking.
- Store CA and entity private keys in an HSM or OS-level secure keystore rather than in process memory.
- Migrate from RSA-2048 to ECDSA/Ed25519 for smaller keys/signatures and lower computational overhead at equivalent security strength.
- Integrate automated certificate renewal (e.g., an ACME-like protocol) to remove manual renewal delay and reduce outage risk from expired certificates.

14. Individual Contribution of Group Members

Team Member	Contribution	Approx. Share
R.SathishKumar (192321148)	PKI architecture design; Root CA setup and self-signed certificate generation	[25%]
K.Senthamizhselvan (192321143)	CSR creation, certificate issuance logic, and CA-side validation implementation	[25%]
K.Dhilip (192365013)	Certificate-based authentication, revocation (CRL) and renewal logic; testing	[25%]
D.Navadeep (192371068)	Report writing, diagrams (architecture/lifecycle), results analysis and SDG mapping	[25%]

15. References

- Stallings, W. "Cryptography and Network Security: Principles and Practice," Pearson Education. (Chapters on Digital Signatures, X.509 Authentication, PKI, and Kerberos.)
- IETF RFC 5280 — "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile."
- IETF RFC 2986 — "PKCS #10: Certification Request Syntax Specification."
- ITU-T Recommendation X.509 — "Information technology – Open Systems Interconnection – The Directory: Public-key and attribute certificate frameworks."
- Python `cryptography` library documentation — https://cryptography.io/en/latest/ (X.509, hazmat primitives).
- United Nations, "Sustainable Development Goals" — https://sdgs.un.org/goals (SDG 9, 16, 17 descriptions).
- Software/tools used: Python 3.12; `cryptography` v46.0.6; Matplotlib (for architecture/lifecycle diagrams); Microsoft Word (report).

16. One-Page Individual Reflections

Reflection – Team Member 1 (PKI Architecture & Root CA)

R.SathishKumar (192321148)

My part of this assignment was designing the overall PKI architecture and implementing the Root Certificate Authority. Before this exercise, I understood a CA as an abstract "trusted third party" from lecture slides; building the self-signed Root CA certificate in code — setting BasicConstraints to mark it as a CA, adding KeyUsage flags for key_cert_sign and crl_sign, and generating its RSA-2048 key pair — made the concept concrete. I had to think carefully about which entity should sit where in the trust hierarchy, and why the CA's certificate has to be self-signed rather than signed by yet another authority (the trust has to terminate somewhere).

The biggest challenge was deciding how much of a production-grade hierarchy to model. Real organizations rarely trust a single flat CA directly — they use an offline Root CA and one or more online Intermediate CAs so that a compromise of a day-to-day issuing CA does not invalidate the entire trust anchor. I chose a single-tier CA for this demonstration to keep the scope manageable, but documenting that as an explicit limitation (Section 13.2) taught me more about production PKI design than simply implementing the two-tier version blindly would have.

This part of the work connects most strongly to SDG 9 (Industry, Innovation and Infrastructure), because the CA is literally the piece of infrastructure that makes every other secure digital interaction in the system possible — without a trustworthy root, nothing downstream can be trusted either. In terms of course outcomes, this gave me a very direct, hands-on application of CO3: I had to reason about digital signatures, certificate structure, and authentication frameworks not as exam definitions but as design decisions with real security consequences.

Reflection – Team Member 2 (CSR Creation & Certificate Issuance)

K.Senthamizhselvan (192321143)

I worked on the entity-side certificate request flow: generating key pairs for the client and server, building their Certificate Signing Requests, and writing the CA-side logic that validates a CSR before issuing a certificate. What surprised me most was realizing that a CSR is itself a signed object — the requesting entity signs its own request with its own private key, purely to prove that it actually possesses that key and is not just submitting someone else's public key. Before implementing this, I had not appreciated that identity binding starts at the request stage, not only at the certificate-issuance stage.

The trickiest part was getting the issuance function to correctly carry forward the CSR's Subject Alternative Name extension into the final certificate, rather than silently dropping it. It was a good reminder that in security-critical code, a missing extension is not a cosmetic bug — an application that only checks the SAN field during hostname verification would silently treat a wrongly-issued certificate as valid for the wrong name, which is exactly the kind of subtle flaw that leads to real-world MITM vulnerabilities. Writing test case TC-03 to explicitly check the issued certificate's subject and serial number gave me more confidence in the correctness of this stage.

This work relates to SDG 16 (Peace, Justice and Strong Institutions), since a CSR-issuance pipeline that verifies identity properly before granting a credential is a small-scale model of the kind of accountable institutional process that prevents fraud and impersonation at larger scale. From a course-outcome perspective, this task let me apply CO3 directly — verifying a CSR's signature and constructing an X.509v3 certificate required understanding exactly how hash-then-sign digital signature schemes work, not just being able to define them.

Reflection – Team Member 3 (Authentication, Revocation & Renewal)

K.Dhilip (192365013)

My contribution covered the "active" half of the system: certificate-based mutual authentication, revocation via a CRL, and certificate renewal. The part I found most conceptually important was separating certificate validation (is this certificate genuine, unexpired, and not revoked?) from proof-of-possession (does the entity presenting this certificate actually control the matching private key?). Initially my authentication function only checked the certificate itself, which meant an attacker who somehow obtained a copy of someone else's public certificate — without their private key — would still have passed my early, incomplete version of the check.

Writing the impersonation test case (Test Case TC-06) forced me to fix this: I added a random-challenge step where the verifier generates a nonce and the peer must sign it, and only a signature that verifies against the certificate's public key counts as authentication. Implementing revocation surfaced a related insight — a revoked certificate can still pass every cryptographic check (correct issuer, valid signature, unexpired), which is exactly why the CRL/OCSP check has to be a mandatory, independent step rather than something bolted on only when convenient. Seeing Test Case TC-08 correctly reject a revoked client even though its certificate was otherwise perfectly valid was, for me, the most satisfying result in the whole assignment.

This part of the work connects to SDG 16, since timely revocation of a compromised credential is precisely the accountability mechanism that limits harm when a private key is stolen. On the course-outcome side, this task engaged CO3 (digital signatures and authentication frameworks) most directly, and also touched CO5, since maintaining and checking a CRL is conceptually similar to how an IDS/IPS maintains and checks a blocklist of known-bad indicators in real time.

Reflection – Team Member 4 (Documentation, Analysis & SDG Mapping)

D.Navadeep (192371068)

My contribution was less about writing the cryptographic code and more about making the whole system legible: producing the architecture and lifecycle diagrams, organizing the test-case and results tables, and researching how this assignment connects to the mapped SDGs and course outcomes. I found that building the architecture diagram (Figure 1) actually deepened my understanding of the code — to draw an accurate picture of message flow between the Client, Server, CA and Certificate Repository, I had to trace through the other members' implementation carefully enough to be sure I wasn't misrepresenting which entity does what.

The most challenging part was the comparative analysis between X.509/PKI and Kerberos (Section 11.2). It would have been easy to just list generic differences from a textbook, but I wanted the comparison to be justified by our specific scenario — a multinational organization authenticating both its own employees and independent partner organizations. Working through why a symmetric, KDC-centric model like Kerberos doesn't scale well across independent organizations (each partner would need to pre-share a secret with a common KDC) helped me understand why asymmetric, CA-anchored trust is the more natural fit here, rather than just asserting that PKI is "better."

This part of the work maps most clearly onto SDG 17 (Partnerships for the Goals), since documenting how a shared CA-based trust model enables secure collaboration between otherwise-independent organizations was the core of the comparative analysis I contributed. From a course-outcome perspective, synthesizing the team's results into a coherent evaluation against authentication strength, scalability, and MITM-resistance gave me a broad, integrative view across CO3, CO4 and CO5, rather than depth in just one narrow implementation task.